

Credit Card Fraud Detection

Shaila Magline J



Introduction

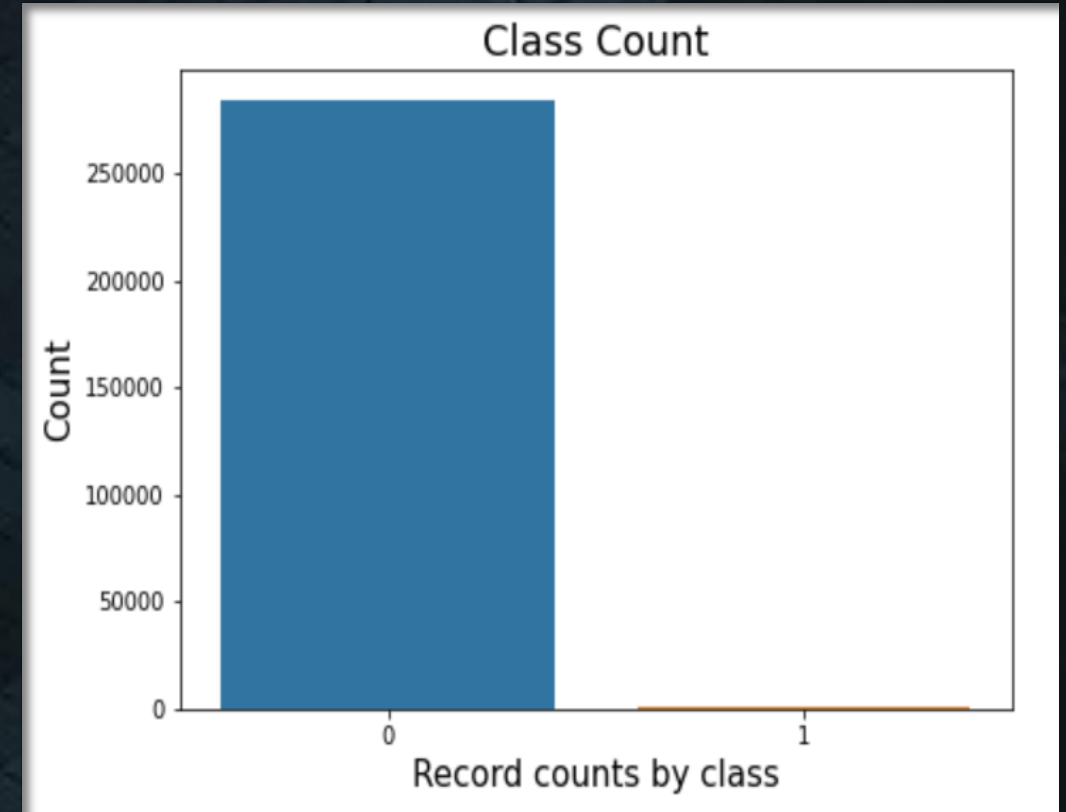


- ❑ **Objective:** Identify the best machine learning model for fraud detection.
- ❑ **Challenge:** Handling class imbalance in fraud detection.
- ❑ **Approach:**
 - Experiment with various oversampling techniques.
 - Evaluate multiple models using StratifiedKFold Cross-Validation.
 - Hyperparameter tuning for the best model.

Dataset Overview



- ❑ **Dataset:** Credit Card Fraud Detection
- ❑ **Imbalance:** Highly skewed data with very few fraudulent transactions.
- ❑ **Features:**
 - Numerical features extracted from transactions.
 - A binary target variable: Fraud (1) or No Fraud (0).



Data Preprocessing



❑ Handling Imbalance:

- Applied Random Oversampling, SMOTE, and ADASYN.
- Used StratifiedKFold Cross-Validation to maintain class distribution.

❑ Feature Engineering:

- Identified most important features.
- Scaled features for better model performance.

```
#Lets perform RepeatedKFold and check the results
from sklearn.model_selection import RepeatedKFold
rkf = RepeatedKFold(n_splits=5, n_repeats=10, random_state=None)
# X is the feature set and y is the target
for train_index, test_index in rkf.split(X,y):
    print("TRAIN:", train_index, "TEST:", test_index)
    X_train_cv, X_test_cv = X.iloc[train_index], X.iloc[test_index]
    y_train_cv, y_test_cv = y.iloc[train_index], y.iloc[test_index]
```


Model Selection



❑ Models Evaluated:

- Logistic Regression
- Decision Trees
- Random Forest
- XGBoost
- Neural Networks

❑ Evaluation Metrics of Best Model:

Model	Oversampling Technique	Accuracy	ROC AUC	Precision	Recall	F1-Score
Logistic Regression	None (Baseline)	99.87%	0.9823	0.94	0.76	0.84
XGBoost	SMOTE	99.89%	0.9792	0.96	0.88	0.92
XGBoost	ADASYN	99.88%	0.9781	0.95	0.86	0.90
Neural Network	SMOTE	99.82%	0.9728	0.91	0.80	0.85

Hyperparameter Tuning for XGBoost



- ❑ Used **RandomizedSearchCV** for optimal hyperparameters.

- ❑ **Best Parameters Found:**

- Max Depth: 7
- Learning Rate: 0.125
- Min Child Weight: 5
- Subsample: 0.8

- ❑ **Final Model Results:**

Model	Oversampling Technique	Accuracy	ROC AUC	Precision	Recall	F1-Score
XGBoost	Random Oversampling	99.93%	0.9815	0.98	0.94	0.96
Random Forest	Random Oversampling	99.85%	0.9754	0.92	0.82	0.87

Conclusion



- ❑ **XGBoost with Random Oversampling & StratifiedKFold CV** gave the best performance after tuning.
- ❑ However, **Logistic Regression with L2 Regularization** (without oversampling) performed best overall in terms of robustness.
- ❑ **Key Takeaway:**
 - Handling class imbalance is crucial.
 - Oversampling + XGBoost = Best for recall.
 - Logistic Regression = Best for balanced performance.

Future Scope



- ❑ Further **Hyperparameter Optimization** for XGBoost.
- ❑ Test with other **resampling techniques** like SMOTE+ENN.
- ❑ **Deploy the model** using an API for real-time fraud detection.

THANK YOU!

